CityTech TTP Summer Bootcamp

Introductions, Environment Setup, and GitHub

2026-06-01

Agenda

1. git Setup and Repo Review
2. Introductions
3. Range Finding
4. Review
5. Environment Setup
6. Intro to Git
7. Exercise

git Setup and Repo Review

- Get onto WiFi
- Create / sign into GitHub
- Bootcamp repo is here: https://github.com/jonathan-chin/citytech-ttpr-2026-summer
- Follow along!

Introductions

- What is TTP?
- Who am I?
- Who are the TAs?
- Who is the Program Director?

This or That? — Pick a side and defend it

- 🍎 Mac or 🪟 Windows?
- 🧊 Iced coffee or ♨️ Hot coffee?
- 🎤 Karaoke or 🤦 Hard pass?
- 🏙️ Manhattan or 🌉 Brooklyn?
- 🌶️ Spicy or 😌 Mild?
- 📖 Read the book or 🎧 Audiobook?

This or That? — Favorites and hot takes

- 🦸 Marvel or 🦇 DC?
- 🌯 Burrito or 🌮 Tacos?
- ⚽ Soccer or 🏀 Basketball?
- 🦆 Horse-sized duck or 🐴 100 duck-sized horses?
- 🎮 Nintendo or 🟦 PlayStation?

Range Finding

Let's see what you already know — shout it out

Range Finding

- What is an **SSH key**?
- What are **Copilot, Claude, and Gemini**?
- What is **git**?

Range Finding

- What is the difference between an **app**, a **web app**, and a **hybrid app**?
- What is **responsive design**?
- What is **Web Inspector / DevTools**?

Range Finding

- What is an **API**, in general?
- What is an **OAuth token**?
- What is **input sanitation**?

Range Finding

- What is a **database**?
- What is **SQL**?
- What is a **server**?

Range Finding

- What is an **SPA** (single-page application)?
- What is a **component**, in React for example?
- What is **CORS**?

Range Finding

- What is **web scraping**?
- What is **exploratory data analysis**?
- What is **PII**?

Range Finding

- What are **3 kinds of data visualization charts**?

Range Finding

- What is a **password hash**?
- What is **authentication**?
- What is **authorization**?

Review

- What do you think we'll be learning this summer?
- What are you most excited for?

Environment Setup

Let's get your machines ready — follow along

> **macOS** — start with Homebrew · **Windows** — start by enabling WSL2

1 · Homebrew (macOS package manager)

The easiest way to install most of today's tools on a Mac.

- Paste into Terminal:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

- **Windows** — no Homebrew; set up WSL2 next (slide 2), then use `apt` inside WSL.

Guide: https://brew.sh

2 · WSL2 (Windows)

WSL (Windows Subsystem for Linux) runs a real Linux environment — Ubuntu — right on Windows, giving you the same Unix shell, tools, and `apt` package manager that macOS and Linux users have.

- **Windows** — in an **Admin PowerShell**, then reboot:

```
wsl --install
```

From here on, run your command-line installs inside the WSL (Ubuntu) terminal — follow the macOS/Linux steps, not PowerShell or Windows `.exe` installers.

Guide: https://learn.microsoft.com/windows/wsl/install

3 · Node, nvm and Yarn

Use **nvm** to manage Node versions; **Yarn** ships with Node via Corepack.

- Install nvm, then:

```
nvm install --lts && corepack enable
```

- **Windows** — run these inside **WSL** (the same Linux steps).

nvm: https://github.com/nvm-sh/nvm

Yarn / Corepack: https://yarnpkg.com/getting-started/install

4 · Git

- **macOS** — `brew install git`
- **Windows** — inside **WSL**, follow the Linux install: `sudo apt install git`

Verify with `git --version`, then set your identity:

```
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"
```

Linux / install guide: https://git-scm.com/download/linux

5 · VS Code

Our editor for the whole bootcamp.

- **macOS** — `brew install --cask visual-studio-code` or download the `.dmg`
- **Windows** — run the installer, then add the **WSL** extension to work inside Linux

Recommended extensions: **WSL**, **GitHub Copilot**, **ESLint**, **Prettier**, **Python**.

Download: https://code.visualstudio.com/download

6.1 · SSH Key — Do you already have one?

Lets you push to GitHub without passwords. **Check before making a new one.**

- List existing keys:

```
ls -al ~/.ssh
```

- Look for a **public** key file like `id_ed25519.pub` or `id_rsa.pub`.
- If you see one, you can reuse it — skip to adding it to GitHub on the next slide.

Checking for keys: https://docs.github.com/en/authentication/connecting-to-github-with-ssh/checking-for-existing-ssh-keys

6.2 · SSH Key — Create one and add to GitHub

- Generate a key (press Enter to accept the defaults):

```
ssh-keygen -t ed25519 -C "you@example.com"
```

- Copy the **public** key:

```
# macOS
pbcopy < ~/.ssh/id_ed25519.pub
# Linux / WSL — print it, then copy the output
cat ~/.ssh/id_ed25519.pub
```

- Paste it into **GitHub → Settings → SSH and GPG keys → New SSH key**.

Generating and adding a key: https://docs.github.com/en/authentication/connecting-to-github-with-ssh

7 · Insomnia (API client)

For testing APIs.

- **macOS** — `brew install --cask insomnia` or download
- **Windows** — run the installer

Download: https://insomnia.rest/download

8 · PostgreSQL

Our database.

- **macOS** — `brew install postgresql@16` (or **Postgres.app**)
- **Windows** — run the **EDB installer** (bundles the server + tools)

Download: https://www.postgresql.org/download/

9 · pgAdmin

A GUI for browsing and querying PostgreSQL.

- **macOS** — `brew install --cask pgadmin4` or download
- **Windows** — run the installer (or it's bundled with the EDB installer above)

Download: https://www.pgadmin.org/download/

10 · Python 3

For data science.

- **macOS** — `brew install python`
- **Windows** — inside **WSL**, follow the Linux install: `sudo apt install python3 python3-pip`

Verify with `python3 --version`.

Download / install guide: https://www.python.org/downloads/

11 · Jupyter

Notebooks for data exploration and visualization.

Once Python is installed (Windows: inside **WSL**):

```
pip3 install jupyterlab
jupyter lab
```

Guide: https://jupyter.org/install

12 · Microsoft 365 and Copilot

- Log in to **Microsoft 365** with your **CUNY credentials**
- Make sure you can access **Copilot** — tell it "Hello World"

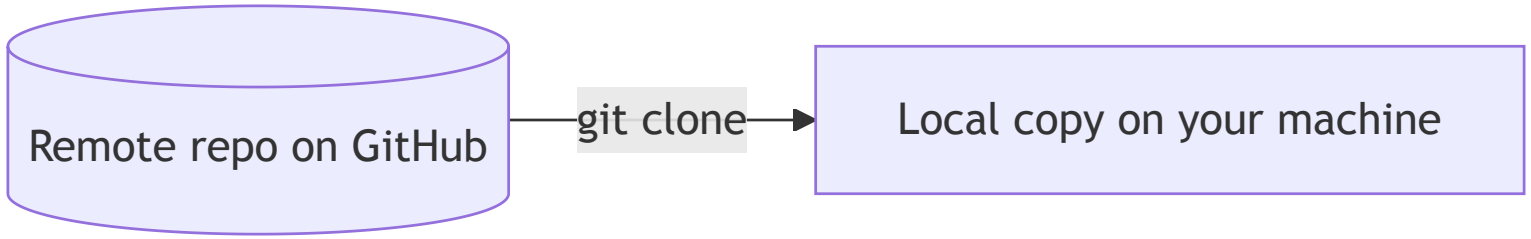
CUNY Microsoft 365: https://www.cuny.edu/about/administration/offices/cis/technology-services/microsoft-office-365-for-education/
Sign in: https://www.microsoft365.com
Copilot: https://copilot.microsoft.com

Intro to Git

Git is a distributed version control system that tracks every change to your code and lets a whole team collaborate without overwriting each other's work. GitHub is a service that hosts your Git repositories online, adding pull requests, issues, and collaboration tools on top.

Clone

Cloning downloads a full copy of a remote repository — every file and its complete history — onto your machine. You'll do this once to start working on a project.




```
git clone git@github.com:org/repo.git
```

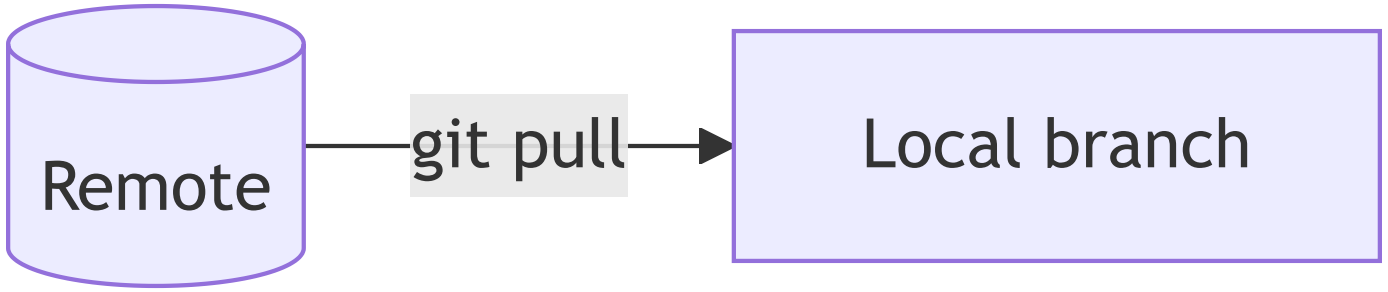
Fork

A fork is your own copy of someone else's repository, made on GitHub. It lets you experiment or propose changes without needing write access to the original (upstream) repo.


```
gh repo fork org/repo   # or click "Fork" on GitHub
```

Pull

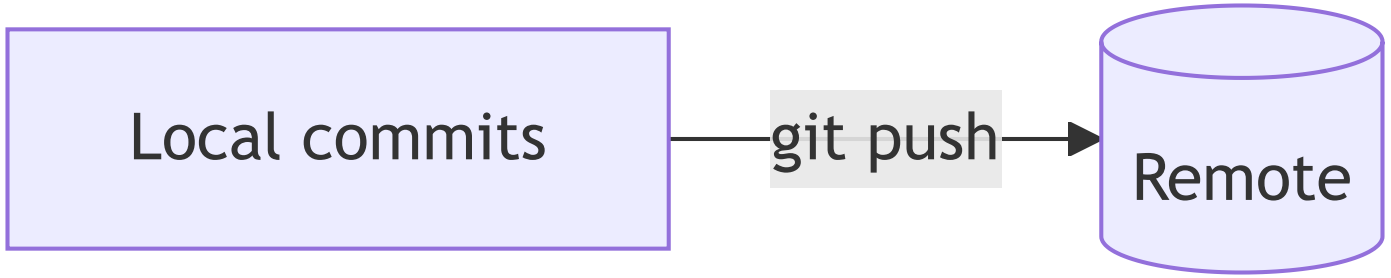
Pulling fetches the latest commits from the remote and merges them into your current local branch. Run it often so your copy stays in sync with your teammates.




```
git pull origin main
```

Push

Pushing uploads your local commits to the remote repository so others can see them. It's how your work leaves your machine and reaches GitHub.




```
git push origin main
```

Exercise

21 Questions over git

https://github.com/jonathan-chin/github_practice_21_questions